

Presentation Agenda

What we'll cover in this training session

01

Intro to Testing

02

SDLC vs STLC

03

STLC Deep Dive

04

Types of Testing

05

Writing Test Cases

06

Bug Management

07

Test Environments

08

Hands-On Practice

09

Tools Overview

10

Mindset & Recap

SECTION 01

Introduction to Software Testing

What it is, why it matters, and what happens when it's skipped

What is Software Testing?

The art and science of finding problems before your users do

Software Testing is the process of evaluating and verifying that software does what it is supposed to do — and does NOT do what it shouldn't.



Verification

- Does the software work correctly?
- Are all requirements implemented?
- Do functions produce correct output?



Validation

- Does it meet user expectations?
- Is it fit for purpose?
- Would the customer accept it?



Quality Assurance

- Prevent defects early
- Improve processes over time
- Build confidence in the product

Why is Testing Important?

The real-world cost of skipping quality checks

\$2.08T

Annual cost of poor software quality (US, 2020)

85%

Of defects introduced in requirements & design phases

100x

More expensive to fix a bug in production vs. requirements

REAL-WORLD FAILURES

NASA Mars Orbiter (1999): Metric vs imperial unit mismatch — spacecraft lost. Cost: \$327M

Therac-25 Radiation (1985): Software race condition caused radiation overdoses, killing patients

Knight Capital (2012): A software bug lost \$440M in 45 minutes on the stock market

Facebook Outage (2021): A config change took down all Facebook services for 6+ hours

SECTION 02

SDLC vs STLC

Understanding the two lifecycles and how they work together

SDLC vs STLC — Key Differences

Two lifecycles that work hand-in-hand

SDLC — Software Development Life Cycle

Covers the ENTIRE development process from idea to product

1. Requirements
2. System Design
3. Implementation
4. Testing
5. Deployment
6. Maintenance

VS

STLC — Software Testing Life Cycle

Focuses ONLY on the testing activities within SDLC

1. Requirement Analysis
2. Test Planning
3. Test Case Development
4. Test Environment Setup
5. Test Execution
6. Test Closure

SECTION 03

Software Testing Life Cycle (STLC)

A deep dive into all 6 phases — activities, deliverables & examples

STLC — The 6 Phases at a Glance

Each phase has clear entry/exit criteria and deliverables



KEY PRINCIPLE: Testing starts from Requirements — NOT from coding!

Entry Criteria

Conditions that must be MET before a phase begins.

Exit Criteria

Conditions that must be met to COMPLETE a phase.

Phase 1: Requirement Analysis

Understanding WHAT needs to be tested

PHASE 01



Objective

- Understand what the software must do
- Identify what needs to be tested
- Determine testable & non-testable requirements
- Identify testing scope and priorities



Key Activities

- Review Business Requirement Docs (BRDs)
- Meet with developers and BAs to clarify
- Identify ambiguous or missing requirements
- Create Requirement Traceability Matrix (RTM)
- Prioritise features by business impact



Deliverables

- Requirement Traceability Matrix (RTM)
- List of ambiguities and questions
- Feasibility report for automation
- Approved test scope document

 Real Example: 'User can log in' — a good tester asks: Wrong password? Max attempts? Locked accounts? Reset flow?

Phase 2: Test Planning

Creating the strategy and roadmap for testing

PHASE 02

Scope

What WILL and WON'T be tested

Objectives

Goals and quality criteria to meet

Resources

Team members, roles, responsibilities

Schedule

Timeline, milestones, deadlines

Risks

What could go wrong and mitigation plans

Tools

Selenium, Playwright, Eclipse, IntelliJ

KEY DELIVERABLE: *Test Plan Document — the 'game plan' that guides the entire testing effort.*

 *Real Example: E-commerce site — 'Test checkout, payment, search. NOT admin panel. Aug 1–14. Two testers. Playwright used for automation.'*

Phase 3: Test Case Development

Translating requirements into testable scenarios

PHASE 03



Key Activities

- Write detailed test cases from requirements
- Create positive AND negative test scenarios
- Prepare test data (valid & invalid inputs)
- Get test cases reviewed by peers
- Map test cases to requirements (RTM)



Deliverables

- Completed test cases (all fields filled)
- Test scripts for Playwright/Selenium automation
- Test data sets (valid & invalid)
- Updated RTM with test case references
- Peer-reviewed and approved test suite



Positive Testing

- Enter valid username and password → Login succeeds
- Add item to cart → Cart count increases



Negative Testing

- Enter wrong password → Error message shown
- Leave username empty → Validation error appears

Phase 4: Test Environment Setup

Preparing the 'testing lab' before execution begins

PHASE 04



Key Activities

- Set up dedicated QA server/environment
- Install required OS, browsers, devices
- Configure databases with test data
- Deploy the application build to QA
- Install Selenium, Playwright, Eclipse, IntelliJ
- Verify the environment is stable and ready
- Smoke test the environment before full testing

Environment Types:

DEV

Developers write code here. Unstable.

QA

Testers work here. Stable build for testing.

UAT

Business users validate. Close to production.

PROD

Live environment. Real users. No testing here!

Phase 5: Test Execution

Running the tests and recording results

PHASE 05



Key Activities

- Execute test cases one by one
- Compare actual vs expected results
- Log PASS or FAIL for each test
- Report defects for all FAIL tests
- Perform retesting after bug fixes
- Conduct regression testing

Test Execution Statuses:

PASS

Actual result matches expected result

FAIL

Actual result does NOT match — raise a defect

BLOCKED

Cannot execute — dependency or environment issue

SKIPPED

Test skipped due to scope change or time

N/A

Test case not applicable in this build



Rule: NEVER mark a test as PASS if you're not 100% sure. When in doubt, mark it BLOCKED or FAIL and raise a query.

Phase 6: Test Closure

Wrapping up, reporting results, capturing lessons learned

PHASE 06



Key Activities

- Evaluate exit criteria — is testing complete?
- Archive all test artefacts (plans, cases, results)
- Write the final Test Summary Report
- Conduct defect analysis (trends, root causes)
- Hold retrospective: What went well? What to improve?
- Get sign-off from project stakeholders
- Share lessons learned with the team



Test Summary Report Includes:

- Total test cases: Passed / Failed / Blocked
- Total defects: Open / Closed / Deferred
- Test coverage achieved (%)



Exit Criteria Example:

- 95%+ test cases executed
- No critical/blocker bugs open
- All high-priority bugs resolved
- Stakeholder approval obtained

 *Real Example: After testing a banking app — 150 tests run, 147 passed, 3 deferred. 12 bugs found, 11 fixed. Sign-off granted.*

SECTION 04

Types of Testing

An overview of testing types every beginner must know

Functional vs Non-Functional Testing

Testing WHAT the system does vs HOW WELL it does it



FUNCTIONAL TESTING

Tests WHAT the system does

- Does the login button work?
- Does checkout process complete?
- Are calculations correct?
- Are error messages shown correctly?
- Examples: Unit, Integration, System, UAT



NON-FUNCTIONAL TESTING

Tests HOW WELL the system works

- How fast does the page load?
- Can it handle 10,000 users at once?
- Is it secure against SQL injection?
- Is it accessible for screen readers?
- Examples: Performance, Security, Usability

Manual vs Automation & Key Testing Types

Knowing when to use each approach



Manual Testing

- Human tester executes test cases step by step
- Best for: Exploratory, usability, ad-hoc testing
- Good for new features with changing requirements



Automation Testing

- Scripts run tests automatically (Selenium / Playwright)
- Best for: Regression, repetitive, data-driven tests
- Uses Eclipse or IntelliJ as the development IDE



Smoke Testing

Quick check that CORE functionality works after a new build. 'Is the app alive?' If smoke fails, stop all further testing — the build is broken.



Regression Testing

Re-run existing tests after code changes to ensure new code hasn't broken OLD functionality. Run as an automated Playwright or Selenium suite.



UAT (User Acceptance)

Business users validate the software meets their needs before go-live. The final gate before production deployment.

SECTION 05

Writing Test Cases

The most critical practical skill for every software tester

What is a Test Case?

The fundamental unit of testing — a step-by-step verification

A Test Case is a set of conditions, inputs, and steps that a tester executes to verify that a specific feature or function of the application works as expected.



A Good Test Case Is...

- Clear — anyone can understand and execute it
- Atomic — tests ONE specific behaviour
- Repeatable — same result every time
- Traceable — linked to a requirement
- Independent — doesn't rely on other test results



A Bad Test Case Is...

- Vague — 'Test the login page' (too broad!)
- Missing expected results — how do you know PASS/FAIL?
- Dependent on order — run test 10 before test 3
- Missing test data — what username/password?
- Not linked to any requirement

Test Case Template

The standard structure used by professional testers

Test Case ID	TC_LOGIN_001
Test Scenario	Verify successful login with valid credentials
Preconditions	User is registered. Browser is open. App is running.
Test Steps	1. Go to login page 2. Enter username 3. Enter password 4. Click Login
Test Data	Username: testuser@demo.com Password: Test@1234
Expected Result	User is redirected to Dashboard. Welcome message shown.
Actual Result	(Filled during execution — leave blank when writing)
Status	PASS / FAIL / BLOCKED / SKIPPED
Comments	Any notes, screenshots, or defect ID references

Test Case Example #1 — Login Page

Worked example: positive and negative scenarios

TC_LOGIN_001 | Valid Login — Positive Test

Preconditions	App is running. User account exists: testuser@demo.com / Test@1234
Test Steps	1. Open login page 2. Enter email: testuser@demo.com 3. Enter password: Test@1234 4. Click Login
Expected	User lands on Dashboard. 'Welcome, Test User!' message is visible.
Status	PASS 

TC_LOGIN_002 | Invalid Password — Negative Test

Preconditions	App is running. User account: testuser@demo.com exists.
Test Steps	1. Open login page 2. Enter email: testuser@demo.com 3. Enter password: WrongPass 4. Click Login
Expected	Login fails. Error message: 'Incorrect password. Please try again.' No navigation.
Status	PASS 

Test Case Example #2 — Registration Form

Testing a sign-up form with multiple validation rules

TC_REG_001 — Register with all valid inputs

PASS

Data: Name: Jane Doe | Email: jane@test.com | Password: Secure@99

Expected: Account created. Confirmation email sent. Redirect to dashboard.

TC_REG_002 — Register with existing email

FAIL ✖

Data: Email: existing@test.com (already registered)

Expected: Error: 'This email is already registered. Please log in.'

TC_REG_003 — Submit form with empty required fields

PASS

Data: All fields left blank

Expected: Form not submitted. Validation messages shown on all required fields.

SECTION 06

Bug / Defect Management

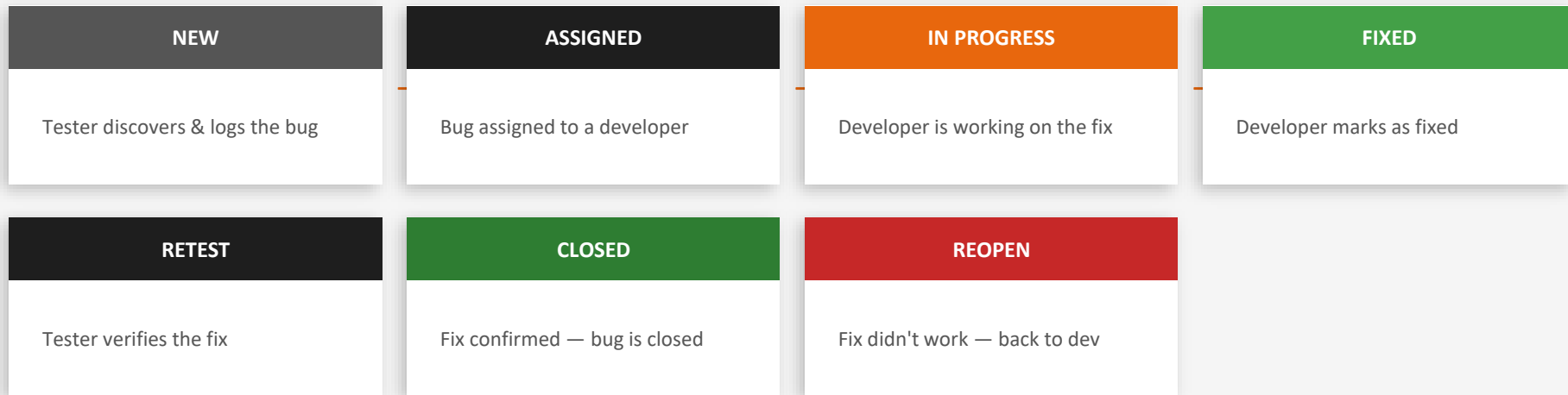
Identifying, reporting, and tracking defects like a professional

Bug / Defect Management

From discovery to resolution — the complete defect journey

A Bug (Defect) is any deviation from the expected behaviour of the software as defined in the requirements or design. If the software does something it shouldn't — or fails to do something it should — that's a bug.

THE BUG LIFECYCLE:



Severity vs Priority

Two different dimensions for classifying defects

Severity — Technical Impact

- How badly does this bug affect the system?
- Critical: App crashes, data lost, security breach
- High: Major feature completely broken
- Medium: Feature works but with workaround
- Low: Minor UI issue, cosmetic defect

Priority — Business Urgency

- How URGENTLY should this bug be fixed?
- Critical: Must be fixed NOW before release
- High: Fix in this sprint/release
- Medium: Can be fixed in next sprint
- Low: Fix when time permits

REAL-WORLD EXAMPLES:



High



High

Bank transfer fails — crash. Fix immediately!



High



Low

Rarely-used admin report crashes — low visibility



Low



High

CEO's name misspelled on homepage — high visibility!



Low



Low

Button colour slightly off on the 404 error page

Bug Report Template

Professional defect report structure used at Lubanzi

Bug ID	BUG_LOGIN_007
Title / Summary	Login fails with valid credentials on Safari browser
Severity	Critical
Priority	High
Environment	QA Server Safari 16.4 macOS Ventura
Build / Version	v2.3.1 — Build #450
Steps to Reproduce	1. Open Safari 2. Navigate to /login 3. Enter valid credentials 4. Click Login 5. Observe error
Expected Result	User logged in and redirected to Dashboard
Actual Result	Error: '500 Internal Server Error'. User NOT logged in.
Attachments	screenshot_bug007.png console_log.txt
Reported By	Jane Intern 2024-08-15

SECTION 07

Test Environments

Understanding where and how testing happens

Test Environments Explained

The right tests in the right environment at the right time



DEV
(Development)

Developers

- Code is written here
- Very unstable — changes every hour
- Dev unit tests run here
- NOT suitable for formal testing



QA
(Testing)

QA Testers

- Stable build deployed here
- Full STLC testing happens here
- Run Selenium & Playwright suites here
- Bug reporting and retesting



UAT
(Staging)

Business Users

- Business validates here
- Realistic data and scenarios
- User Acceptance Testing
- Final check before go-live



PROD
(Production)

End Users

- Live system — real users!
- NEVER test here directly
- Monitored for errors 24/7
- Hotfixes only in emergencies



Golden Rule: Run Selenium/Playwright suites in QA. A bug verified in QA must go through a proper release process before production.

SECTION 09

Tools Overview

The tools we use at Lubanzi every day

Automation Tools: Selenium & Playwright

The two primary automation frameworks used at Lubanzi



Selenium — Web Browser Automation

The industry-standard open-source framework for automating web browser interactions. Widely used in the QA industry.

- Automates clicking, typing, and navigation in real browsers (Chrome, Firefox, Edge)
- Supports Java (Eclipse/IntelliJ), Python, JavaScript, C# — great for regression suites
- Works with TestNG and JUnit frameworks for structured test execution



Playwright — Modern Cross-Browser Automation

Microsoft's modern automation framework. Faster, more reliable, and supports all major browsers out of the box.

- Supports Chromium, Firefox, and WebKit (Safari) — true cross-browser testing in one tool
- Auto-waits for elements — fewer flaky tests. Built-in tracing and screenshot capture
- Write tests in JavaScript/TypeScript, Python, Java (IntelliJ), or C# (.NET)

IDE Tools: Eclipse & IntelliJ

Your development environment for writing automation code



Eclipse IDE

Free & open-source Java IDE by the Eclipse Foundation

- Industry standard for Java-based Selenium projects
- Excellent Maven and Gradle integration for dependency management
- TestNG plugin for structured test suite organisation
- Built-in debugger to step through automation scripts
- Free to use — ideal for learning and intern projects
- Large community and extensive plugin ecosystem



IntelliJ IDEA

Smart Java IDE by JetBrains — Community edition is free

- Excellent Playwright Java and Selenium support out of the box
- Superior code completion and intelligent refactoring
- Integrated test runner for TestNG/JUnit with rich output
- Built-in Git integration for version control
- Community edition is free — Ultimate adds advanced features
- Preferred by many professional teams for productivity

SECTION 10

The Tester Mindset

The habits and attitudes that make a great QA engineer

The QA Tester's Mindset

Skills and habits that separate good testers from great ones



Curiosity

Always ask 'What if?' and 'What happens when?' Challenge every assumption.



Critical Thinking

Think like a user AND an attacker. How would someone misuse this feature?



Attention to Detail

Notice what others miss. One wrong decimal can cause a banking disaster.



Communication

Write clear bug reports. Communicate findings without blaming developers.



Persistence

Don't give up after one test. Explore every path, every edge case.



Continuous Learning

Tech changes fast — stay updated on Selenium, Playwright and new tools.

"Testing is not about finding bugs — it is about delivering confidence."

Questions Every Good Tester Asks

Critical thinking in action — real questions for every project

Clarify Scope	? What is this feature supposed to do?
Null Inputs	? What happens if the user enters nothing?
Boundary Testing	? What is the maximum allowed input length?
Security	? What happens after 5 failed login attempts?
Compatibility	? Does this work on mobile browsers?
Resilience	? What happens if the network drops mid-transaction?
Usability	? Is the error message clear enough for a user?
Automation	? Can this be automated with Playwright or Selenium?
Concurrency	? Can two users do this simultaneously without conflict?

Summary & Key Takeaways

Everything you learned today — condensed into one slide

1

Software Testing finds defects BEFORE users do — it saves money, reputation, and sometimes lives

2

STLC has 6 phases: Requirement Analysis → Planning → Development → Environment → Execution → Closure

3

Test cases must be clear, specific, traceable, and include both positive AND negative scenarios

4

Every failed test = a bug report with Severity, Priority, Steps to Reproduce, and Expected vs Actual

5

Always test in the correct environment — QA for testing, NEVER directly in Production

6

At Lubanzi we automate using Selenium and Playwright, developed in Eclipse and IntelliJ

7

The best testers are curious, detail-oriented communicators who always ask 'What if?'

THE BIG ANALOGY

Testing Software is Like Testing a Car Before Selling It

STLC Phase	 Car Testing Equivalent
Requirement Analysis	Reading the car's specification sheet — what must it do?
Test Planning	Planning the test drive route, who drives, what to check
Test Case Development	Writing a checklist: brakes, wipers, horn, AC, fuel, speed
Environment Setup	Fuelling up, checking tyre pressure, getting to the test track
Test Execution	Actually driving and checking each item on the list
Test Closure	Signing the QA report — car passed/failed, issues documented